操作系统复习笔记 - 完整版

> 教材: Operating Systems
> 教授: Prof. Yongkun Li (USTC)
> 整理日期: 2026-06-29

第一章 操作系统概述 (Chapter 1: Overview of OS)

1.1 计算机系统组织

中断处理 (Interrupt Handling)

类型	触发方式
硬件中断 (Hardware interrupt)	硬件发送信号给CPU
软件中断 (Software interrupt / Exception / Trap)	软件执行特殊操作：系统调用 (system call)

中断过程:

1. CPU停止当前工作
2. 执行中断服务例程 (Interrupt Service Routine, ISR)
3. CPU恢复之前的工作

> **关键原则**: 操作系统是中断驱动的 (Operating system is interrupt driven)

- **中断向量 (Interrupt Vector)**: 指向中断例程的指针表，存储在**低内存 (low memory)**中
- 中断架构必须保存被中断指令的地址
- 现代架构将返回地址存储在**系统栈 (system stack)**上

存储层次结构 (Storage Hierarchy)

基于三个因素: **速度 (Speed)**、**成本 (Cost)**、**易失性 (Volatility)**

层次	特点
主存 (Main Memory)	CPU可直接访问的唯一大容量存储；随机访问，易失 (volatile)
二级存储 (Secondary Storage)	大容量非易失性 (nonvolatile)；硬盘(磁道/扇区)、SSD

缓存 (Caching)

- 将信息复制到更快的存储系统中；主存可视为二级存储的缓存
- 先检查缓存，若有则直接使用；若无则从慢存储复制到缓存
- **缓存管理**: 缓存大小 (Cache size) + 替换策略 (Replacement policy)

直接内存访问 (DMA)

- 设备控制器将数据块从缓冲区**直接传输到主存，无需CPU干预**
- 每个块只产生一次中断，而不是每字节一次中断

计算机系统架构

架构	特点
单处理器系统	一个主CPU执行通用指令集
多处理器系统 (SMP)	增加吞吐量、规模经济、增加可靠性(优雅降级/容错)
ASMP (非对称多处理)	主从关系 (boss-worker)，每个处理器分配特定任务
SMP (对称多处理)	所有处理器平等 (peers)，每个处理器执行所有任务
NUMA	非一致内存访问，添加CPU导致
多核 (Multicore)	单芯片包含多个核心，片上通信比片间通信更快，更低功耗
集群系统 (Clustered)	多个独立系统通过SAN共享存储，提供高可用性

集群类型:

- 非对称集群: 一台热备模式 (hot-standby)
- 对称集群: 多节点运行应用程序并互相监控
- 使用**分布式锁管理器 (DLM)** 避免冲突操作

1.2 操作系统的定义与目标

计算机系统的四个组件

组件	描述
用户 (Users)	人、机器、其他计算机
硬件 (Hardware)	CPU、内存、I/O设备
应用程序 (Application Programs)	字处理器、编译器、Web浏览器等
操作系统 (OS)	控制和协调各应用程序和用户之间对硬件的使用

OS目标

1. 执行用户程序，使解决用户问题更容易
2. 使计算机系统使用更方便
3. 以高效方式使用计算机硬件
4. **设计权衡**: 便利性与效率之间的权衡

OS的四种视角

视角	描述
系统视角	OS是资源分配器 (resource allocator) + 控制程序 (control program)
PC用户	追求便利、易用和良好性能，不关心资源利用率
共享计算机	必须让所有用户满意，最大化资源利用率
移动/嵌入式	资源贫乏，优化可用性和电池寿命

OS定义

- **内核 (Kernel)**: "计算机上一直运行的那一个程序"
- 其他: **系统程序 (system program)** + **应用程序 (application program)**

1.3 操作系统操作

多道程序设计 (Multiprogramming)

- OS最重要的方面，提高效率
- 组织作业使得CPU始终有一个可执行
- 作业池 (job pool) 在磁盘上，部分子集在内存中
- 当作业等待(如I/O)时，OS切换到另一个作业

多任务/分时 (Multitasking / Time Sharing)

- 多道程序设计的逻辑扩展
- CPU频繁切换作业，用户可交互 → **交互式计算 (interactive computing)**
- **响应时间 (Response time)**: 应 < 1秒

关键问题	解决方案
多个作业同时准备好运行	**CPU调度 (CPU scheduling)**
进程不适合内存	**交换 (swapping)**
进程不完全在内存中	**虚拟内存 (Virtual memory)**

双模式操作 (Dual-mode Operation)

模式	描述
用户模式 (User mode)	运行用户代码
内核模式 (Kernel mode)	运行操作系统代码

- **模式位 (Mode bit)**: 由硬件提供，区分用户代码或内核代码
- **特权指令 (Privileged instructions)**: 只能在内核模式下执行
- **系统调用 (System call)**: 将模式切换到内核模式，返回时重置为用户模式

系统调用 (System Calls)

- **定义**: 进程和OS内核之间的编程接口 (programming interface)
- 通常采用**陷阱 (trap)** 到中断向量中特定位置的形式
- 特征: **原始的 (primitive)**、**重要的 (important)**、**基础的 (fundamental)**

系统调用 vs. 库函数调用:

对比项	系统调用	库函数调用
实现位置	操作系统内核内	库文件中
示例	`open()`	`fopen()`
说明	原始、底层	封装系统调用，更友好

OS 标准:

标准	全称	示例OS
POSIX	Portable Operating System Interface	Linux
BSD	Berkeley Software Distribution	Mac OS Darwin
SVR4	System V Release 4	Solaris Unix

1.4 操作系统组件

进程管理 (Process Management)

- **程序 (Program)**: 被动实体 (passive entity)，存储在磁盘上的文件
- **进程 (Process)**: 活动实体 (active entity)，程序的一个执行实例
- 进程是系统内的工作单元 (unit of work)
- **单线程进程**: 一个程序计数器；**多线程进程**: 每个线程一个程序计数器

进程管理活动:

1. 创建和删除用户和系统进程
2. 挂起和恢复进程
3. 提供进程同步机制
4. 提供进程通信机制
5. 提供死锁处理机制

内存管理 (Memory Management)

C程序内存布局 (Segmentation):

```
高地址
   ┌─────────────┐
   │    Stack    │ ← 局部变量 (Local variables) ↓
   │     ↓       │
   │     ↑       │
   │    Heap     │ ← 动态分配的内存
   ├─────────────┤
   │ Data Segment│ ← 全局变量 (Global variables) + 常量
   ├─────────────┤
   │ Text Segment│ ← 程序代码 (Program code)
   └─────────────┘
低地址
```

内存管理活动:

1. 跟踪内存的哪些部分正在被使用以及被谁使用
2. 决定哪些进程（或其部分）和数据应移入或移出内存
3. 根据需要分配和释放内存空间

存储管理 (Storage Management)

- OS提供统一的、逻辑的信息存储视图
- **抽象**: 将物理属性抽象为逻辑存储单元 → **文件 (file)**

文件系统必须记录的内容: 目录、文件、已分配空间、空闲空间

基本文件操作: 打开(Open)、读取(Read)、写入(Write)、关闭(Close)、重命名(Rename)、删除(Delete)

> 注意: 创建不是基本操作(是打开的特殊情况)；复制不是基本操作；移动等同于重命名(除非跨磁盘)

1.5 内核数据结构

数据结构	原则/特点	用途
单向链表	每个节点指向下一个节点	-
双向链表	每个节点指向前一个和后一个节点	Linux进程表
循环链表	最后一个节点指向第一个节点	-
栈 (Stack)	后进先出 (LIFO)	函数调用
队列 (Queue)	先进先出 (FIFO)	作业调度
二叉搜索树 (BST)	左子树 <= 右子树，最坏 O(n)	-
平衡BST	如红黑树，最坏 O(log n)	Linux CFS调度器
哈希映射 (Hash Map)	数据检索 O(1)，需处理哈希冲突	-
位图 (Bitmap)	n个二进制数字表示n个项目的状态，空间效率高	磁盘块可用性

1.6 计算环境

环境	描述
传统计算	独立通用机器，互联网互联
移动计算	手持智能手机、平板；Apple iOS 和 Google Android
分布式计算	分离的、可能异构的系统通过网络连接；LAN/WAN/MAN/PAN
客户端-服务器	服务器响应客户端生成的请求
P2P	不区分客户端和服务器，所有节点视为对等点
虚拟化	允许OS在其他OS内运行应用程序；VMM (Virtual Machine Manager)
云计算	SaaS/PaaS/IaaS；公有云/私有云/混合云
实时嵌入式	具有明确定义的固定时间约束；Hard real-time / Soft real-time

关键术语对照表 (Ch1)

中文	English	中文	English
操作系统	Operating System (OS)	内核	Kernel
进程	Process	程序	Program
中断	Interrupt	中断向量	Interrupt Vector
系统调用	System Call	陷阱	Trap
用户模式	User Mode	内核模式	Kernel Mode
多道程序设计	Multiprogramming	多任务/分时	Multitasking / Time Sharing
虚拟内存	Virtual Memory	DMA	Direct Memory Access
对称多处理	SMP	非一致内存访问	NUMA
集群系统	Clustered Systems	虚拟机管理器	VMM
保护	Protection	安全	Security

第二章 操作系统结构 (Chapter 2: OS Structures)

2.1 操作系统服务 (OS Services)

面向用户的服务

服务	内容
程序执行 (Program execution)	将程序装入内存、运行程序、结束执行
I/O操作 (I/O operations)	read, write等；用户通常不能直接控制I/O设备
文件系统操作 (File-system manipulation)	读写文件和目录、创建删除、搜索、权限管理
通信 (Communications)	共享内存 (Shared memory) 或 消息传递 (Message passing)
错误检测 (Error detection)	CPU错误、内存错误、I/O设备错误、用户程序错误
用户接口 (User interface)	CLI (命令行)、Batch (批处理)、GUI (图形用户界面)

确保效率的服务

服务	内容
资源分配 (Resource allocation)	CPU周期、主存、文件存储、I/O设备
记账 (Accounting)	跟踪用户使用了多少/何种资源
保护与安全 (Protection and security)	控制资源访问、用户认证、防御外部攻击

2.2 系统调用 (System Calls)

参数传递三种方法

1. **寄存器传递 (Registers)**: 最简单，参数可能超过寄存器数量
2. **表方式 (Table-based)**: 参数存储在内存块中，块地址作为寄存器参数传递 (Linux/Solaris)
3. **栈方式 (Stack-based)**: 程序将参数压入栈，OS从栈中弹出

六大类别系统调用

类别	主要操作
进程控制 (Process Control)	`fork()`, `exec()`, `exit()`, `wait()`, `abort()`
文件管理 (File Management)	`open()`, `close()`, `read()`, `write()`, `create`, `delete`
设备管理 (Device Management)	`request device`, `release device`, `read`, `write`
信息维护 (Information Maintenance)	获取/设置时间、系统数据、调试(内存转储)
通信 (Communications)	消息传递模型 / 共享内存模型

类别	主要操作
保护 (Protection)	`get_permission()`, `set_permission()`, `allow_user()`, `deny_user()`

2.3 操作系统结构

结构类型	描述	优点/缺点
简单结构 (MS-DOS)	在最小空间提供最多功能； 接口和功能层次未分离	应用程序可直接访问基本I/O例程； 易受错误程序影响
单体内核 (UNIX)	内核包含系统调用接口以下的所有内容	难以实现和维护；**性能优势**
层次式 (Layered)	OS分为多个层，每层构建在低层之上	易于构建和调试；**效率问题**
微内核 (Mach)	尽可能将功能从内核移到用户空间； 只提供最小进程管理、内存管理和通信	易于扩展/移植、更可靠安全；**性能开销**(用户空间↔内核空间通信)
模块 (Modules)	可加载内核模块(LKM)；内核有核心组件， 通过模块链接额外服务	灵活、高效；大多数现代OS使用
混合系统 (Hybrid)	组合不同结构	Linux=单体内核+模块化; Windows=单体内核+微内核子系统; Mac OS X=Mach微内核+BSD Unix

机制与策略分离 (Mechanism vs Policy) - 重要原则

- **Mechanism (机制)**: How to do it?（如何做？）
- **Policy (策略)**: What will be done?（做什么？）
- **好处**: 最大灵活性；可在不改变机制的情况下改变策略

2.4 调试、生成与引导

- **日志文件 (log files)**: 进程失败时写入错误信息
- **核心转储 (core dump)**: 进程内存的快照
- **崩溃转储 (crash dump)**: OS崩溃时的内存状态
- **Kernighan's Law**: "调试的难度是编写代码的两倍"
- **SYSGEN**: 操作系统生成，为特定计算机站点配置OS
- **Bootstrap program (引导程序)**: 驻留在ROM中；定位内核、装入内存并启动；常见: GRUB

第三章 进程概念与操作 (Chapter 3: Process

Concepts & Operations)

3.1 进程概念

程序 vs 进程

对比项	程序 (Program)	进程 (Process)
本质	**被动实体 (passive entity)**	**活动实体 (active entity)**
存储	磁盘上的可执行文件	内存中的执行实例
关系	一个程序可对应多个进程	一个进程可运行多个程序(exec)

程序构建流程

1. **预处理 (Pre-processor)**: 展开 `#define`，`#include`，`#ifdef`
2. **编译与优化 (Compiler & Optimizer)**: 语法检查、构建汇编代码
3. **汇编 (Assembler)**: 将 `.s` 汇编为 `.o` 目标代码
4. **链接 (Linker)**: 将所有 `.o` 文件与库链接
    - **静态库 (Static library)**: `.a` 文件，最终程序是代码的组合
    - **动态库 (Dynamic/Shared library)**: `.so` 文件，链接器只检查函数是否存在

进程在内存中的布局

段	内容
Text section (代码段)	程序代码
Data section (数据段)	全局变量
Stack (栈)	临时数据（函数参数、返回地址、局部变量）
Heap (堆)	进程运行时动态分配的内存
Program counter (程序计数器)	下一条指令的位置
CPU registers (CPU寄存器)	寄存器的内容

进程状态 (Process State)

```
New → Ready → Running → Terminated
         ↑        ↓
         └ Waiting ↵
```

状态	描述
new (新建)	进程正在被创建
running (运行)	指令正在被执行
waiting (等待)	进程正在等待某个事件发生

状态	描述
ready (就绪)	进程正在等待被分配给处理器
terminated (终止)	进程已完成执行

> **关键**: 在任何时刻，一个处理器上只能有一个进程在运行

PCB (Process Control Block) / task_struct

PCB字段	描述
Process state	running, waiting 等
Program counter	下一条指令的位置
CPU registers	所有进程相关寄存器的内容
CPU scheduling information	优先级、调度队列指针
Memory-management information	分配给进程的内存
I/O status information	分配给进程的I/O设备、打开的文件列表
Accounting information	使用的CPU时间、时间限制

进程层次结构

- **init 进程**: PID=1, 内核启动时创建的第一个进程
- **孤儿进程 (Orphan)**: 父进程终止时子进程成为孤儿；Linux中init进程成为所有孤儿的继母 (re-parent)
- **查看进程树**: `pstree` 或 `pstree -A`

3.2 进程操作

fork() - 创建进程

- fork() 调用一次，返回**两次**（父进程和子进程中分别返回）
- **父进程**: fork() 返回值为子进程的PID
- **子进程**: fork() 返回值为0

fork() 克隆的内容:

克隆项	描述
Program code	共享同一代码段
Memory	局部变量、全局变量、动态分配的内存
Opened files	父进程打开的文件子进程也自动打开
Program counter	两者从fork()返回后同一行代码开始执行

fork() 不克隆的内容:

区别项	父进程	子进程
fork()返回值	子进程的PID	0
PID	不变	新值
Running time	累积	刚创建，为0

exec*() - 执行程序

- 6个成员: `execl()`, `execv()`, `execle()`, `execve()`, `execlp()`, `execvp()`
- **丢弃**: 内存（局部变量、全局变量、动态分配内存）、寄存器值（如程序计数器）
- **保留**: PID、进程关系、运行时间等
- exec()调用后，原来程序中的后续代码不会被执行

wait() - 等待子进程

- `wait()` 挂起调用进程直到某个子进程终止
- 如果没有正在运行的子进程，wait()不会挂起
- **wait() 的返回值**: 被终止子进程的PID

特性	wait()	waitpid()
等待范围	等待任一子进程	可等待特定子进程
检测状态	仅检测子进程终止	可检测子进程状态变化

fork() + exec*() + wait() = system() 的标准实现

3.3 内核视角的进程操作

内核空间 vs 用户空间

特性	Kernel-space memory	User-space memory
存储内容	内核数据结构、内核代码、设备驱动程序	进程的内存、进程的程序代码
访问者	内核代码	用户程序代码 + 内核代码

用户时间 vs 系统时间

- **User time**: 花费在用户空间内存中代码上的时间
- **System time**: 花费在内核空间内存中代码上的时间
- **Total running time** = user time + system time
- 测量: `time` 命令

fork() 内部工作机制

Step 1 (Kernel-space): 复制PCB → 更新子进程PCB (PID、运行时间、父进程指针等) → 插入任务链表
Step 2 (User-space): 复制父进程用户空间的所有内容

Step 3: 分别在父进程和子进程中返回

exec*() 内部工作机制

1. 搜索目标程序文件
2. 清除用户空间内存
3. 更新寄存器
4. 保留PCB中的PID、进程关系等

exit() + wait() 机制

exit() 步骤:

1. 清理大部分已分配的内核空间内存
2. 清理所有用户空间内存
3. 向父进程发送 **SIGCHLD** 信号

僵尸进程 (Zombie Process):

- 子进程调用exit()后变为僵尸
- 不再运行，但PCB条目保留在进程表中
- **无法杀死僵尸进程**（因为它已经死了）
- Linux中标记为 `<defunct>`

wait() 的作用:

- **wait() 和 exit() 一起使用!**
- **wait() 用于回收(reap)僵尸子进程**
- 僵尸占用PID，PID总数有限 (查看: `cat /proc/sys/kernel/pid_max`)

第四章 线程 (Chapter 4: Threads)

4.1 线程概念

为什么使用线程

- 进程创建是 heavyweight (重量级) 的
- 许多数据可以在多个任务间共享
- 现代计算机包含多核，CPU未被充分利用

并发 vs 并行

概念	说明
Concurrency (并发)	单核系统上的交错执行
Parallelism (并行)	多核系统上的同时执行

多线程在内存中的结构

- **Code**: 所有线程共享相同的代码
- **Global variables & Dynamic memory**: 所有线程共享
- **Local variables**: 每个线程有自己的私有栈 (stack)

多线程的好处

1. **Responsiveness (响应性)**: 当一个线程被阻塞时，进程可依赖另一个线程
2. **Ease in data sharing (数据共享简便)**: 通过全局变量和动态分配内存实现共享
3. **Economy (经济性)**: 创建线程比创建进程快数十倍；上下文切换更快
4. **Scalability (可扩展性)**: 线程可在不同核心上并行运行

编程挑战

1. Identifying tasks (任务识别)
2. Balance (平衡)
3. Data splitting (数据分割)
4. Data dependency (数据依赖) → 需要同步
5. Testing and debugging (测试与调试)

4.2 线程模型

三种线程模型

模型	描述	优点	缺点
Many-to-One (多对一)	所有线程映射到内核中的一个进程结构	内核容易实现	一个阻塞系统调用会阻塞所有线程
One-to-One (一对一)	每个线程映射到一个进程/线程结构	阻塞只影响调用线程； 高并发度	受限于内核内存大小
Many-to-Many (多对多)	多个线程映射到多个结构(组映射)	可创建所需数量的线程； 高并发度	-

- Linux和Windows使用 **One-to-One** 模型

4.3 Pthreads API

操作	进程	线程
Creation	`fork()`	`pthread_create()`
ID Type	PID (整数)	`pthread_t` (结构体)
Who am I	`getpid()`	`pthread_self()`
Termination	`exit()`	`pthread_exit()`
Wait for child	`wait()` / `waitpid()`	`pthread_join()`
Kill	`kill()`	`pthread_kill()`

4.4 隐式线程 (Implicit Threading)

线程池 (Thread Pools)

- **思想**: 创建一个线程池，线程在其中等待工作
- **优点**:
  i. 使用现有线程服务请求比创建新线程快
  ii. 将应用中的线程数绑定到池的大小

OpenMP

- 为共享内存环境中的并行编程提供支持
- 一组编译器指令和API
- `#pragma omp parallel for` — 创建与处理核心数量相同的线程

4.5 线程相关问题

fork() 和 exec() 的语义

- `fork()`: 某些UNIX系统有两种版本 — 复制所有线程 或 只复制调用fork()的线程
- `exec()`: 替换运行中的进程 — 包括所有线程

线程取消 (Thread Cancellation)

方式	描述
Asynchronous cancellation (异步取消)	立即终止目标线程
Deferred cancellation (延迟取消)	允许目标线程定期检查是否应该被取消 (默认模式)

线程本地存储 (TLS)

- 允许每个线程拥有自己的数据副本，跨函数调用可见，类似于静态数据但是每个线程独有的

第五章 进程通信与同步 (Chapter 5: Process Communication & Synchronization)

5.1 进程通信 (IPC)

独立进程 vs 协作进程

- **Independent process**: 不能影响或被其他进程影响
- **Cooperating process**: 可以影响或被其他进程影响

IPC 的两个抽象模型

模型	描述	特点
Shared Memory (共享内存)	建立共享内存区域，读写共享区域	更快；被视为常规内存访问
Message Passing (消息传递)	交换消息	需要内核干预；更容易在分布式系统中实现

共享内存 - 生产者-消费者问题

- **Buffer (缓冲区)**: 允许进程并发运行
  - Bounded buffer (有界缓冲区) / Unbounded buffer (无界缓冲区)
- `in == out` 表示缓冲区空
- `(in+1) % BUFFER_SIZE == out` 表示缓冲区满

消息传递 - 实现问题

1. **Naming (命名)**: 直接通信(对称/非对称) vs 间接通信(mailbox/端口)
2. **Synchronization (同步)**: 阻塞(同步) vs 非阻塞(异步)
   - **Rendezvous (会合)**: send和receive都是阻塞时
3. **Buffering (缓冲)**: Zero capacity / Bounded capacity / Unbounded capacity

Socket (套接字)

- 通信的端点；由 IP地址 + 端口号 标识
- 1024以下端口用于标准服务 (Telnet:23, FTP:21, HTTP:80)
- **127.0.0.1 (loopback)**: 允许同一主机上的客户端和服务器使用TCP/IP通信
- 三种类型: TCP (面向连接)、UDP (无连接)、Multicast (多播)

Pipe (管道)

类型	特点
Ordinary pipes (普通管道)	无文件系统名称；仅相关进程使用(父子关系)；同一机器；单向；通信完成后消失
Named pipes (命名管道/FIFO)	有文件系统名称；无需父子关系；同一机器；多个进程可使用；双向(半双工)

5.2 竞争条件 (Race Condition)

- **定义**: 执行结果取决于共享资源被访问的特定顺序
- **根本原因**: 高级操作不是原子 (atomic) 的
  - 底层汇编: `load → modify → write` 三条指令可能被中断
- **要点**: 竞争条件总是不好的；99%执行正常，1%有问题；调试非常困难
- **解决方案**: 需要进程同步 (Process Synchronization)

5.3 互斥 (Mutual Exclusion)

临界区 (Critical Section)

- **定义**: 访问共享对象的代码段
- **结构**: Entry section → Critical section → Exit section → Remainder section
- **目标**: 当一个进程在其临界区执行时，不允许其他进程进入其临界区

临界区设计的四项要求

#	要求	含义
1	**Mutual Exclusion (互斥)**	不能有两个进程同时在临界区内
2	**相对速度无假设**	每个进程以非零速度执行，对相对速度和CPU数量不做假设
3	**Progress (前进)**	没有在临界区外运行的进程应阻塞其他进程
4	**Bounded Waiting (有限等待)**	没有进程会永远等待进入临界区 (无饥饿)

5.4 互斥实现方案

方案一: 禁用中断 (Disabling Interrupts)

- 进程在临界区内时禁用上下文切换
- **问题**: 多处理器环境中不可行；性能问题(牺牲并发性)

方案二: 互斥锁 (Mutex Locks) / 自旋锁 (Spinlock)

```
void acquire() {
    while (!available)
        ; /* busy waiting */
    available = false;
}
void release() {
    available = true;
}
```

- **问题**: Busy waiting (忙等待) 浪费CPU资源
- **应用**: 多处理器系统中锁预期持有时间很短时

方案三: Peterson 解决方案

- 共享数据: `int turn;` + `boolean interested[2];`
- 进程像绅士一样行动: "如果你想进入，我让你先"
- **问题**: 忙等待；潜在优先级反转问题 (priority inversion)

方案四: 信号量 (Semaphore) - 最终方案

- **两种类型**:
  - **Binary semaphore (二值信号量)**: 0 或 1 (类似互斥锁)
  - **Counting semaphore (计数信号量)**: 控制有限数量的资源
- **两个标准原子操作**:
  - `down()` / `P()` / `wait()`: 递减计数，count=0时阻塞
  - `up()` / `V()` / `signal()`: 递增计数，唤醒等待进程

解决忙等待: 使用阻塞队列代替忙等待

解决原子性: 禁用中断来保证down()和up()的原子性，临界区尽可能短

信号量用途:

1. Mutual exclusion (互斥): 使用二值信号量
2. Counting (计数): 使用计数信号量
3. Process synchronization (进程同步): 可能需要多个信号量

5.5 死锁 (Deadlock)

死锁的四个必要条件

#	条件	描述
1	**Mutual Exclusion (互斥)**	一次只有一个进程可以使用资源
2	**Hold and Wait (持有并等待)**	进程持有至少一个资源并等待获取其他资源
3	**No Preemption (不可抢占)**	资源只能由持有它的进程自愿释放

#	条件	描述
4	**Circular Wait (循环等待)**	存在等待进程集合形成循环等待链

资源分配图 (Resource-Allocation Graph)

- 无环 → 无死锁
- 有环 → 每资源类型单实例: 死锁；每资源类型多实例: 可能死锁

银行家算法 (Banker's Algorithm)

- **Safe state (安全状态)**: 存在一个运行序列保证所有进程的需求可以满足
- **Unsafe state (不安全状态)**: 不存在任何序列满足需求 (不等于死锁，但可能最终导致死锁)
- 对每个请求: Safe → 接受；Unsafe → 拒绝

死锁处理策略

1. **检测并恢复**: 使用资源分配图检测环的存在
2. **预防/避免**: 银行家算法
3. **忽略**: 假装死锁从未发生 — 大多数OS（包括UNIX和Windows）使用此方法

5.6 经典IPC问题

哲学家就餐问题 (Dining Philosopher Problem)

- 5位哲学家，5根筷子；吃饭需恰好两根筷子
- **目标**: 构造同步协议使不会死锁且不会饿死
- **关键思想**: 不需要实际的筷子对象，只需要保证哲学家i在吃时，左右邻居不能吃
- **信号量方案**: 1个mutex(保护state数组) + N个sN

生产者-消费者问题 (有界缓冲区)

三个信号量的角色:

信号量	初始值	作用
`mutex`	1	互斥访问缓冲区
`empty`	N	空槽位计数，为0时生产者睡眠
`full`	0	已占槽位计数，为0时消费者睡眠

> **重要**: 不能交换 `down(&mutex)` 和 `down(&empty)` 的顺序，否则可能导致死锁!

读者-写者问题 (Reader-Writer Problem)

- 规则:
  i. 读者正在读时，其他读者允许读
  ii. 读者正在读时，不允许写者写

iii. 写者正在写时，不允许其他写者和读者访问

- **读者优先于写者**: 只要还有读者，写者就必须被阻塞
- 使用信号量: `db` (保护数据库互斥访问) + `mutex` (保护read_count)

第六章 进程调度 (Chapter 6: Process Scheduling)

6.1 基本概念

进程执行模型

- 进程执行 = CPU突发 (CPU burst) + I/O突发 (I/O burst) 的循环
- 多道程序设计/多任务: 提高CPU利用率和系统响应性

上下文切换 (Context Switching)

- **上下文 (Context)**: 进程的用户空间内存和寄存器值的联合
- **步骤**: 放弃CPU → 备份寄存器到PCB → 调度器选择下一进程 → 加载新进程上下文
- **代价高**: 目标进程可能存储在硬盘中；**最小化上下文切换次数**有助于提高性能

6.2 调度准则 (Scheduling Criteria)

调度类别

类别	描述	适用
Non-preemptive (非抢占式)	进程不离开调度器直到自愿等待I/O或释放CPU	强调任务完成时间的系统
Preemptive (抢占式)	进程可能因特定中断和事件被强制离开CPU	强调交互性的系统；现代OS

性能度量指标

指标	定义	目标
CPU utilization (CPU利用率)	保持CPU尽可能忙碌	越高越好
Throughput (吞吐量)	每时间单位完成的进程数	越高越好
Turnaround time (周转时间)	从提交到完成的时间间隔	越低越好
Waiting time (等待时间)	在就绪队列中等待的时间	越低越好
Response time (响应时间)	从提交请求到产生第一个响应的时间	越低越好

6.3 调度算法

FCFS (先来先服务)

- **非抢占**: 按到达顺序服务
- **问题**: 护航效应 (Convoy effect) — 一个长进程阻塞后面的短进程
- 敏感于输入顺序

SJF (最短作业优先)

- **非抢占式SJF**: 选择具有最短CPU需求的进程
- **最优性**: SJF被证明是给出最小平均等待时间的最优算法
- **抢占式SJF (SRTF)**: 当新进程到达时，根据剩余CPU需求选择下一个任务
- **挑战**: 如何知道下一个CPU请求的长度?
- **预测**: 指数平均 `Predicted = α × last_burst + (1-α) × previous_predicted`

RR (轮转调度)

- **抢占式**: 每个进程被给予一个 **quantum (时间片)**
- 专门为分时系统设计
- 时间片: 10-100ms；经验法则: 80%的CPU突发应短于时间片
- **权衡**: RR vs SJF — RR等待时间/周转时间更长，但**响应性极好**

优先级调度 (Priority Scheduling)

- 调度器基于优先级选择下一个进程
- **问题**: 无限制阻塞或饥饿 (Starvation)
- **解决方案**: Aging (老化) — 逐渐增加等待进程的优先级

多级队列调度 (Multilevel Queue)

- 每个优先级类使用不同的调度算法
- 例: 前台进程(RR) + 后台进程(FCFS)
- **问题**: 固定优先级可能导致饥饿

多级反馈队列调度 (Multilevel Feedback Queue)

- **改进**: 允许进程在队列间移动 (动态优先级)
- **最通用但也最复杂**: 可配置以匹配特定系统

6.4 实时系统调度

类型	描述
Hard real-time (硬实时)	任务必须在其截止时间前被服务
Soft real-time (软实时)	关键进程比非关键进程优先 (无保证)

速率单调调度 (Rate Monotonic)

- 每个进程分配与速率成比例的优先级
- 使用带抢占的静态优先级策略 (固定优先级)
- **问题**: 不能保证一组进程都能被调度

最早截止时间优先 (EDF)

- 根据截止时间动态分配优先级 (截止时间越早，优先级越高)
- 不要求进程是周期性的

6.5 SMP调度问题

概念	描述
Processor Affinity (处理器亲和性)	尝试让进程在同一处理器上运行 (软/硬亲和性)
NUMA	CPU调度器和内存放置算法协同工作
Load Balancing (负载均衡)	Push migration / Pull migration

6.6 Linux CFS 调度器

- 优先级 0-99: 特权类 (实时进程，遵循RR或FIFO)
- 普通进程: **CFS (Completely Fair Scheduler, 完全公平调度器)**
- 每个进程维护 **vruntime** (虚拟运行时间)，CFS选择最小vruntime的进程
- **Decay factor (衰减因子)**: nice值 (-20到+19)，值越小优先级越高
- 使用 **红黑树 (red-black tree)** 维护可运行的任务

第七章 内存管理 (Chapter 7: Memory Management)

7.1 程序员视角 - 地址空间

进程内存布局 (Segmentation)

段 (Segment)	内容	说明
Stack (栈)	局部变量、函数参数	向下增长；默认8MB
Heap (堆)	动态分配的内存 (malloc)	向上增长
Data Segment & BSS	全局变量和静态变量	Data: 已初始化; BSS: 未初始化
Code + Constant (代码段)	程序代码和常量	只读 (read-only)

- **32位系统**: 最大4GB地址空间 (2^32 bytes)
- **64位系统**: 更大

数据段 vs BSS

- **Data**: 存储**已初始化**的全局变量和静态变量；编译时分配空间
- **BSS (Block Started by Symbol)**: 存储**未初始化**的全局变量和静态变量；进程启动时才分配
  - BSS上的大数组不会增加可执行文件大小

栈 (Stack)

- FILO (First In, Last Out) 结构
- 函数调用: 参数压栈 → 返回地址压栈 → 分配局部变量
- 返回: 返回值写入EAX → 栈收缩 → CPU跳回
- 返回后栈内存不归还给OS，被后续函数调用重用
- **栈溢出 (Stack Overflow)**: 无限递归导致

堆 (Heap) / 动态分配

- **malloc()**: 调用 **brk()** 系统调用分配空间
- **free()**:
  - 释放最后一个块: 调用brk()收缩堆
  - 释放中间块: 维护空闲块链表 (记录指针+大小)
  - 32位系统每空闲块开销: 8字节；64位: 16字节

7.2 空闲列表实现

分配策略

策略	描述
First fit (首次适应)	分配第一个足够大的空洞 (快)
Next fit (下次适应)	类似first fit，从上次搜索位置开始
Best fit (最佳适应)	分配最小的足够大的空洞 (减少碎片，搜索更慢)
Worst fit (最差适应)	分配最大的空洞

碎片化 (Fragmentation)

- **外部碎片 (External fragmentation)**: 堆内存有许多空洞 → 解决方法: 压缩 (Compaction)
- **内部碎片 (Internal fragmentation)**: 载荷小于分配的块大小

合并 (Coalescing)

- **[Knuth'73] 边界标记 (Boundary Tag)**: 在页脚添加标签，实现常量时间的合并

空闲列表类型

- **隐式空闲列表 (Implicit Free List)**: 每个块记录size+is_allocated
- **显式空闲列表 (Explicit Free List)**: 只追踪空闲块
- **分离空闲列表 (Segregated Free List)**: 不同大小类别的不同空闲列表
  - **伙伴系统 (Buddy System)**: 2的幂次块大小

7.3 内核视角 - 虚拟内存

为什么需要虚拟内存

1. 进程内存可能增长，连续分配无法满足
2. 进程大小可能超过物理内存大小
3. 地址空间不需要连续
4. 进程可以超过物理内存大小

MMU (Memory Management Unit)

- CPU中的模块，执行**地址转换**: Logical Address → Physical Address

虚拟内存的优点

1. 不同进程使用相同虚拟地址，翻译到不同物理地址
2. 内存共享: 线程共享内存；不同进程共享代码
3. 内存增长: 新分配的内存不需要连续

7.4 分页 (Paging)

基本概念

- **页 (Page)**: 虚拟内存的固定大小块 (通常 **4KB** = 4096 bytes)
- **页框/帧 (Frame)**: 物理内存的固定大小块
- **页表 (Page Table)**: MMU内部的查找表，存储虚拟页到物理页框的映射
- **VPN (Virtual Page Address)**: 高位 (如20 bits)
- **Page Offset**: 低位 (如12 bits)

页表项内容

字段	说明
虚拟页号	隐式索引，不实际存储
权限 (Permission)	rwx (读写执行)，s = sharable
有效-无效位 (Valid-Invalid bit)	1=在内存中，0=不在内存中
页框号 (Frame #)	物理页框的编号

页表结构

- **两层页表 (Two-level Page Table)**: 将页表分成多个部分，避免连续分配大块内存
- **哈希页表 (Hashed Page Tables)**
- **反向页表 (Inverted Page Tables)**

TLB (Translation Lookaside Buffer)

- 页表查找的缓存
- 搜索速度快: 指令流水线的一部分
- 大小小: 32-1024 项

有效内存访问时间: `EAT = hit_ratio × mem_time + miss_ratio × (mem_time + mem_time)`

7.5 请求分页 (Demand Paging)

核心思想

- 分配是**惰性 (lazy)** 的!
- `malloc()` 只分配虚拟地址空间，不分配物理页
- `memset()` 真正访问内存时，才请求系统分配物理页
- 这是**延迟分配 (Lazy Allocation)**

缺页中断 (Page Fault) 处理流程

1. 程序访问虚拟地址 → MMU检查页表
2. 发现页无效 → CPU生成缺页中断
3. 缺页处理程序: 分配内存页 → 更新页表项 (有效位=1)
4. 重新执行触发缺页的指令

交换区 (Swap Area)

- Linux: 交换分区 (swap partition)
- Windows: pagefile.sys
- **最大进程大小 = 物理内存大小 + 交换分区可用空间 - 内核内存大小**

写时复制 (Copy-on-Write, COW)

- fork()后父进程和子进程共享页面
- 只有当其中一个进程想要**写入**共享页面时，才复制和修改一个新页面
- 避免fork()后立即执行exec*()的浪费

缺页性能

- `EAT = (1 - p) × memory_access_time + p × page_fault_time`
- memory_access: 10-200 ns; page_fault: ~8 ms
- 仅允许10%性能下降时，缺页率必须低于 **0.0000025**

7.6 页面置换算法 (Page Replacement Algorithms)

(1) 最优置换算法 (Optimal / MIN)

- **策略**: 置换未来最长时间内不会使用的页
- **优点**: 缺页次数最少，理论上最优
- **缺点**: 需要预知未来 — **不可实现**
- **用途**: 作为性能比较的基准

(2) FIFO (先进先出)

- **策略**: 最先调入内存的页最先被置换出去
- **问题**: **Belady异常** — 增加页框数反而导致缺页增加

(3) LRU (最近最少使用)

- **策略**: 置换最长时间未被访问的页
- **原理**: 基于**局部性原理 (locality rule)** — 最近被访问的页近期可能再次被访问
- **优点**: 性能好，无Belady异常 (栈算法)
- **实现**: 计数器法 / 栈法(双向链表)

(4) 二次机会 / 时钟算法 (Second-Chance / Clock)

- **基础**: FIFO
- **改进**: 检查引用位 (reference bit)
  - 引用位=1 → 清零并给第二次机会
  - 引用位=0 → 换出
- **Clock算法**: 二次机会的环形队列实现
- 如果所有引用位都为1 → 退化为FIFO

Belady异常

- **定义**: 增加页框数反而导致缺页增加的现象
- FIFO算法可能出现Belady异常
- **栈算法 (Stack algorithms)** 不会出现Belady异常: LRU是栈算法

7.7 页框分配与颠簸

分配算法

1. **平均分配 (Equal Allocation)**: 每进程 m/n 个
2. **比例分配 (Proportional Allocation)**: 按进程大小比例分配
3. **基于优先级的方案 (Priority-based Scheme)**

颠簸 (Thrashing)

- **定义**: 进程没有足够的页框 → 频繁缺页 → 系统花在换页上的时间比执行时间更多

- **解决方法**:
  i. 局部置换/优先级置换
  ii. **工作集策略 (Working-Set Strategy)**: 估计所需页框数
    - 工作集: 最近Δ次页引用中访问的页面集合
  iii. **基于缺页频率 (Page-Fault Frequency)**: 监控缺页率并动态调整

7.8 内核内存分配

伙伴系统 (Buddy System)

- 从固定大小的段分配内存
- 2的幂次分配器 (11种order)
- 优点: 合并 (coalescing) 方便

Slab分配 (Slab Allocation)

- 为小对象分配内存 (限制碎片)
- Slab: 一个或多个连续页
- Cache: 一个或多个Slab
- 每个唯一的内核数据结构有一个独立的cache

内存映射文件 (Memory-Mapped File)

- 将文件关联到虚拟地址空间的一部分
- 首次访问: 请求分页；后续读写: 常规内存访问
- 提高性能，允许多进程映射同一文件
- 系统调用: `mmap()`

第八章 大容量存储与I/O系统 (Chapter 8: Mass Storage & I/O)

8.1 磁盘结构

术语	说明
Track (磁道)	盘片表面分为同心圆磁道
Sector (扇区)	磁道分为扇区 (512B数据 + ECC校验)
Cylinder (柱面)	同一磁臂位置下所有磁道的集合

访问时间

- **寻道时间 (Seek time)**: 移动磁臂到目标柱面

- **旋转延迟 (Rotational latency)**: 自旋转到目标扇区 (5400/7200/10K/15K RPM)

两种记录方式

方式	特点	使用
CLV (恒定线速度)	变转速保持相同数据速率	CD-ROM/DVD-ROM
CAV (恒定角速度)	转速恒定	硬盘

8.2 磁盘管理

格式化

1. **低级格式化/物理格式化**: 将磁盘划分为扇区，填充ECC
2. **分区**: 分为一个或多个柱面组
3. **逻辑格式化**: 创建文件系统

坏块管理 (Bad Block Management)

- **扇区备用/重定向 (Sector Sparing/Forwarding)**: 坏扇区逻辑替换为备用扇区
- **扇区滑动 (Sector Slipping)**: 重新映射到下一个扇区

8.3 磁盘调度算法

算法	描述	优点	缺点
FCFS	按请求到达顺序服务	公平	性能差， 摆动大
SSTF	选择与当前磁头位置最近的请求 (SJF形式)	性能好	可能导致饥饿
SCAN (电梯算法)	从一端移到另一端，服务请求， 到达后反转	高负载下性能好， 不易饥饿	远端等待长
C-SCAN (循环扫描)	到达一端后立即返回(不服务返回路径)	更均匀的等待时间	-
LOOK / C-LOOK	只移动到最后的请求位置	磁头移动最少	-

8.4 固态硬盘 (SSD)

闪存类型

类型	每单元位数	寿命	特点
SLC	1位	100K P/E	寿命更长，延迟更低，成本更高
MLC	2位	10K P/E	容量更高
TLC	3位	几千-几百	-

SSD组织结构

```
Package > Die/Chip > Plane > Block > Page
```

操作特性

操作	单位	特点
Read (读)	page (4KB)	~25μs + 100μs总线
Write (写/Program)	page (4KB)	~200μs + 100μs总线
Erase (擦除)	block (64/128 pages)	~1.5ms

- 编程只能 1→0，擦除只能 0→1
- **不能原地覆盖 (No overwrite)**
- 删除仅标记页面为无效 (invalid)

8.5 FTL (Flash Translation Layer)

三个核心功能

1. **地址映射 (Address Mapping)**
2. **垃圾回收 (Garbage Collection)**
3. **磨损均衡 (Wear-Leveling)**

地址映射方案

方案	特点	适用场景
扇区映射 (Sector Mapping)	映射表大，需要大量RAM	随机工作负载
块映射 (Block Mapping)	映射表小，频繁擦除	顺序工作负载
混合映射 (Hybrid Mapping)	先块映射，再扇区映射	-
日志结构化映射 (Log-structured)	数据块:块映射；日志块:扇区映射	大顺序+小随机混合

8.6 RAID (Redundant Array of Independent Disks)

RAID级别	描述	特点
RAID 0	块级条带化，无冗余	高性能，**不提高可靠性**
RAID 1	数据镜像	双倍磁盘，高可靠性
RAID 0+1	先条带化再镜像	与RAID1存储成本相同
RAID 1+0	先镜像再条带化	容错性优于RAID01
RAID 4	专用奇偶校验盘	校验盘可能成为瓶颈
RAID 5	分布式奇偶校验	理想组合:性能、容错、容量、存储效率
RAID 6	双奇偶校验	容忍两个磁盘故障

校验更新方式

- **RMW (Read-Modify-Write)**: 读旧数据+旧校验 → 计算新校验 → 写 (2读+2写)
- **RRW (Read-Reconstruct-Write)**: 读其他数据 → 重构 → 写

纠删码 (Erasure Codes)

- **Cauchy Reed-Solomon (CRS)**: 从k个数据块生成m个编码块；容忍任意m个磁盘故障
- 基于XOR的编码: RDP, EVENODD, X-Code

8.7 I/O系统

I/O控制方式

方式	描述	特点
轮询 (Polling)	CPU循环检查设备状态 (忙等待)	设备快时合理，设备慢时效率低
中断 (Interrupts)	设备触发中断通知CPU	两条线:可屏蔽/不可屏蔽
DMA (直接内存访问)	绕过CPU直接在设备和内存间传输	需要DMA控制器

内核I/O子系统

功能	描述
I/O Scheduling (调度)	维护请求队列，重新排序请求
Buffering (缓冲)	应对设备速度不匹配、传输大小不匹配、维持拷贝语义
Caching (缓存)	快速设备保存数据的副本
Spooling (假脱机)	为一次只能服务一个请求的设备保存输出(如打印)

第九章 文件系统 (Chapter 9: File Systems)

9.1 文件系统基础

层次结构

```
用户进程 → 库函数调用(fopen/fread/fwrite/fclose)
→ 系统调用(open/read/write/close)
→ 内核函数(NTFS/Ext4/FAT32/ISO9660)
→ 设备
```

- **OS ≠ FS**: OS支持FS，一个OS可支持多个FS
- **存储设备 ≠ FS**: 设备只是一个容器

文件 (File)

- **OS视角**: 文件是**逻辑存储单元 (logical storage unit)**，抽象数据类型
- **用户视角**: 文件是**逻辑二级存储器的最小分配单元**
- **文件内容与元数据分离**: 内容在数据块中；文件名在父目录中；大小是属性

文件属性

属性	修改命令	系统调用
名称	`mv`	`rename()`
大小	更新内容	`write()`, `truncate()`
权限	`chmod`	`chmod()`
所有者	`chown`	`chown()`
时间	`touch`	`utime()`

目录 (Directory)

- **目录本质上也是一种文件** — 拥有文件属性和内容
- 目录文件内容记录的是属于该目录的所有文件和子目录
- **文件创建的本质 = 更新目录文件** (添加新目录项)
- **路径名**在整个文件系统中唯一；**文件名**只在所属目录内唯一

9.2 文件操作

基本文件操作

操作	描述
Create (创建)	分配空间，在目录中添加条目
Write (写)	文件名，文件内容 (写指针)
Read (读)	文件名，内存位置 (读指针)
Reposition (重定位)	文件寻址 (不涉及实际I/O)
Delete (删除)	释放空间，擦除目录项
Truncate (截断)	只保留属性

文件描述符 (File Descriptor, fd)

- **fd本质上是一个数组索引**, 用于每个进程定位其打开的文件
- 标准文件描述符: **stdin=0, stdout=1, stderr=2**
- `FILE` 结构包含: fd + 缓冲区 (buffer)

内核缓冲缓存 (Kernel Buffer Cache)

- 提高读/写性能
- 写操作先写入缓存，之后才刷新到磁盘
- 这就是为什么需要"安全弹出"USB

9.3 文件系统布局方案

Trial 1.0: 连续分配 (Contiguous Allocation)

- 文件名 → 起始地址 + 结束地址
- **优点**: 实现简单，容易定位
- **缺点 #1**: 外部碎片 (External Fragmentation)
- **缺点 #2**: 增长问题 (Growth Problem) — 文件无法增长
- **实际应用**: ISO9660 (只读文件系统)

Trial 2.0/2.1: 链表分配 (Linked List Allocation)

- 将存储设备切分为等大小的块 (blocks)
- 每个块借用4字节存储下一块的块号
- **优点**: 解决外部碎片；文件可自由增长
- **缺点 #1**: 随机访问性能问题 — 最坏O(n)次I/O
- **缺点 #2**: 内部碎片 (Internal Fragmentation)

Trial 2.2: FAT (File Allocation Table)

- **核心改进**: 将所有"下一个块号"信息**集中存储**在FAT中
- FAT本质是一个数组，通过指针加法实现随机访问

FAT版本:

文件系统	簇地址大小	最大簇数
FAT12	12 bits	4K
FAT16	16 bits	64K
FAT32	32 bits (28位使用)	256M

FAT条目值:

- `0x00000000`: 未分配 (Unallocated)
- `0x0FFFFFF7`: 坏簇 (Damaged)
- `>= 0x0FFFFFF8`: 文件结束 (EOF)

Trial 3.0: 索引节点分配 (Index-Node Allocation)

- **核心思想**: 将FAT打散成小块，每个文件拥有自己的索引节点
- **固定大小的索引节点**

索引节点结构:

- 12个 Direct Block Pointers (直接块指针)
- 1个 Indirect Block (单级间接块)
- 1个 Double Indirect Block (双级间接块)
- 1个 Triple Indirect Block (三级间接块)

文件大小计算 (主导项: 2^(4x-6)):

- 块大小=1KB → ~16GB
- 块大小=4KB → ~4TB

实际应用: UNIX/Linux的Ext2/3/4文件系统

9.4 文件系统信息与分区

MBR (Master Boot Record, 主引导记录)

- 512字节
- Boot Code: 446字节 — 指定启动哪个分区
- Partition Table: 4个条目，每个16字节
- Signature: 0xAA55, 2字节

分区的优点

1. **性能**: 较小的文件系统更高效
2. **多系统引导 (Multi-booting)**: 单硬盘可安装多个OS
3. **数据管理 (Data Management)**: 不同逻辑驱动器存储不同类型数据

第十章 FAT32 与 Ext2/3 文件系统详解 (Chapter 10)

10.1 FAT32 详解

FAT32 分区典型布局

区域	描述
Boot Sector (引导扇区)	存储FS特定参数，1扇区(512字节)
FSINFO	空闲空间管理，1扇区
Reserved Sectors (保留扇区)	由微软保留
FAT (2份)	FAT2是FAT1的备份
Root Directory (根目录)	目录树的起点，至少1簇

目录项结构 (32字节)

字节偏移	描述
0	文件名的第一个字符 (0x00=未分配, 0xE5=已删除)
1-10	文件名(7字符)+扩展名(3字符) — **8+3命名规范**
11	文件属性 (只读, 隐藏等)
20-21	首簇地址高2字节
26-27	首簇地址低2字节
28-31	文件大小 (最大4GB-1)

大端序 vs 小端序

- **大端序 (Big Endian)**: 高位字节在低地址
- **小端序 (Little Endian)**: 低位字节在低地址
- **FAT使用小端序 (Little Endian)**, 因为基于Intel x86

LFN (Long File Name, 长文件名)

- 通过添加多个额外目录项表示长文件名
- 每个LFN项以Unicode编码(每字符2字节)表示13个字符

- LFN项的Byte 11总是 0x0F
- LFN项的顺序是**倒序的 (descending order)**

FAT32 文件操作

- **删除文件**: 释放簇(FAT标记为0) + 目录项第一字节改为0xE5
- **"已删除"文件的实质**: 数据并未真正移除，只是标记为空闲
- **恢复已删除文件**: 只要扇区未被重新分配和覆写，数据就持续存在

10.2 Ext2/3 详解

Ext2/3 布局 (块组结构)

组结构	描述
Superblock (超级块)	存储FS特定数据
GDT (Group Descriptor Table)	存储块位图、inode位图、inode表的位置
Block Bitmap (块位图)	指示哪些块已分配 (1=已分配, 0=未分配)
Inode Bitmap (Inode位图)	指示哪些inode已分配
Inode Table (Inode表)	按inode号排序的inode数组
Data Blocks (数据块)	存储文件的数据块数组

分组原因

1. **可靠性**: 每个组都有超级块副本
2. **性能**: inode和其数据块物理上更接近 → 局部性原理

Inode 结构 (128字节)

字节偏移	描述
0-1	文件类型和权限
2-3	用户ID (User ID)
4-7	文件大小低32位
8-23	时间信息
26-27	**链接计数 (Link Count)**
40-87	12个直接数据块指针
88-91	单级间接块指针
92-95	双级间接块指针
96-99	三级间接块指针

字节偏移	描述
108-111	文件大小高32位

> **注意**: inode中**不包含文件名** — 文件名存储在目录结构中

目录结构

```
struct dirent {
    ino_t          d_ino;       // inode number
    off_t          d_off;       // offset to the next dirent
    unsigned short  d_reclen;    // record length
    unsigned char   d_type;      // file type
    char           *d_name;      // file name
}
```

硬链接 vs 符号链接

特性	硬链接 (Hard Link)	符号链接 (Symbolic Link)
定义	指向现有inode的目录项	存储目标路径名的特殊文件
inode	相同的inode	新的inode
创建	`ln source target`	`ln -s target link_name`
删除	`unlink()` — 链接计数减1	删除链接文件不影响目标
跨文件系统	不支持	支持

链接计数 (Link Count)

- 普通文件创建时: 链接计数 = **1**
- 目录创建时: 链接计数 = **2** (自身的`.`+父目录中的条目)
- 链接计数降至0时: 内核释放数据块和inode

10.3 内核缓冲缓存

三种缓冲缓存

缓存类型	作用
Page Cache (页缓存)	缓冲打开文件的数据块
Directory Entry Cache (dcache)	内核存储目录项
Inode Cache (Inode缓存)	内核临时存储inode内容

写模式

模式	描述
Write-through (通写)	磁盘副本和缓存同时更新；write()直到磁盘写入完成后才返回
Write-back (回写)	先更新缓存，延迟更新磁盘副本；通过sync/fsync()写回

10.4 日志 (Journaling)

为什么需要日志

- 由于缓冲缓存，系统在掉电或崩溃时可能存在**不一致性**
- 磁盘保证: 每次只原子写入一个扇区

基本思想

- 在覆写磁盘结构之前，先写下描述即将执行操作的"小纸条"
- **Write-ahead Logging (预写日志)**

原子事务

- 一组FS操作成为原子事务: 要么全部成功，要么全不执行
- 每个事务写入日志后再应用到FS

数据日志 (Data Journaling) 操作序列

1. **Journal Write (日志写入)**: 将事务内容写入日志
2. **Journal Commit (日志提交)**: 写入TxE (事务结束块)
3. **Checkpoint (检查点)**: 将更新写入最终磁盘位置

元数据日志 (Metadata Journaling)

模式	描述
Write-back mode (回写模式)	数据和日志写入无顺序约束；元数据可能指向垃圾数据
Ordered mode (有序模式)	数据先写入FS，再提交日志；先写被指向的对象，再写给指向它的对象

- Ext3和NTFS广泛部署**有序模式 (Ordered mode)**

10.5 VFS (Virtual File System, 虚拟文件系统)

- **VFS**: FS抽象层，透明且统一地支持多个文件系统
- VFS定义接口，具体FS实现该接口
- 调用者(VFS层)不需要关心具体使用的是哪个FS
- 不同FS拥有自己的操作集 (如 `fat_file_operations`, `ext3_file_operations`)

附录: 全局核心概念速查表

概念	英文	所属章节	一句话说明
操作系统	Operating System	Ch1	用户与硬件之间的中间人程序
中断	Interrupt	Ch1	CPU停止当前工作执行服务例程
系统调用	System Call	Ch1/2	进程与OS内核的编程接口
进程	Process	Ch3	程序的一个执行实例，活动实体
PCB	Process Control Block	Ch3	内核中表示进程的数据结构
线程	Thread	Ch4	CPU利用的基本单元，轻量级进程
互斥	Mutual Exclusion	Ch5	一次只有一个进程在临界区内
信号量	Semaphore	Ch5	控制资源访问的同步原语
死锁	Deadlock	Ch5	进程集合互相等待对方释放资源
调度	Scheduling	Ch6	选择下一个进程运行
虚拟内存	Virtual Memory	Ch7	逻辑地址到物理地址的映射
分页	Paging	Ch7	固定大小块的地址映射机制
TLB	Translation Lookaside Buffer	Ch7	页表查找的硬件缓存
缺页	Page Fault	Ch7	访问不在物理内存中的页
LRU	Least Recently Used	Ch7	置换最长时间未被访问的页
Belady异常	Belady's Anomaly	Ch7	增加页框反而增加缺页(仅FIFO)
颠簸	Thrashing	Ch7	频繁换页导致系统性能急剧下降
文件系统	File System	Ch9	组织和管理存储设备上数据的系统
FAT	File Allocation Table	Ch9/10	集中式链表，记录簇分配信息
inode	Index Node	Ch9/10	Ext中存储文件元数据的结构
硬链接	Hard Link	Ch10	指向同一inode的多个目录项
符号链接	Symbolic Link	Ch10	存储路径名的特殊文件
日志	Journaling	Ch10	保证崩溃一致性的机制
VFS	Virtual File System	Ch10	FS抽象层，统一支持多种FS

整理自: Operating Systems, Prof. Yongkun Li (USTC), Chapters 1-10